

NYC Taxi Simulated Streaming Pipeline — Migration Design

1. Business objective

Build a restartable, governed streaming pipeline that moves NYC TLC taxi events from PostgreSQL through Kafka and PySpark into curated PostgreSQL analytics tables. The solution must preserve completeness, quarantine invalid data, deduplicate at-least-once delivery, and provide measurable V1/V2 performance evidence.

2. Architecture

PostgreSQL taxi_src.trips → Kafka taxi.trips.raw (3 partitions, trip_key message key) → immutable Bronze NDJSON → PySpark Structured Streaming → Silver Parquet + Quarantine JSON → foreachBatch PostgreSQL fact_trips and Gold aggregates.

3. Delivery and reliability guarantees

Kafka is at-least-once. Producer persists a monotonic trip_id high-water mark. Kafka consumer commits offsets after successful Bronze batch writes. Spark checkpoints preserve source progress and state. PostgreSQL fact load uses COPY into a temporary staging table followed by ON CONFLICT (trip_key) DO NOTHING; Gold uses ON CONFLICT DO UPDATE. Together these provide effectively exactly-once sink behavior.

4. Incremental extraction

Use WHERE trip_id > :last_trip_id ORDER BY trip_id with a server-side cursor. OFFSET is intentionally rejected because it is slower for deep pages and is not a stable high-water mark under concurrent inserts. The checkpoint is checkpoints/producer_offset.json.

5. Validation and quarantine

Rules include non-negative fare/total, distance 0–200 miles, passenger count 0–8, location IDs 1–265, dropoff not before pickup, trip duration ≤12 hours, mandatory identifiers/event time, and January 2024 event-period validation. Every rejected message is written to quarantine with raw_json, reject_reason, and quarantined_at.

6. Deduplication

Use event time tpep_pickup_datetime with a 10-minute watermark and dropDuplicates([trip_key,tpep_pickup_datetime]). The watermark bounds the state store. Silver is partitioned by event-time year and month.

7. V1 vs V2

V1 uses 200 shuffle partitions, one file per trigger, no broadcast join, 16 KB Kafka batches, zero linger and no compression. V2 uses 8 shuffle partitions, 16 files per trigger, broadcast zone join, 64 KB Kafka batches, 50 ms linger and gzip compression. These settings target a constrained local development environment and are benchmarked on the same bounded workload.

8. Schema evolution

config.TRIP_SCHEMA is the event contract and is versioned with SCHEMA_VERSION. Additive fields should be nullable. New optional JSON fields are ignored by older consumers, while missing new fields resolve to null. A production deployment should use Schema Registry compatibility policies.